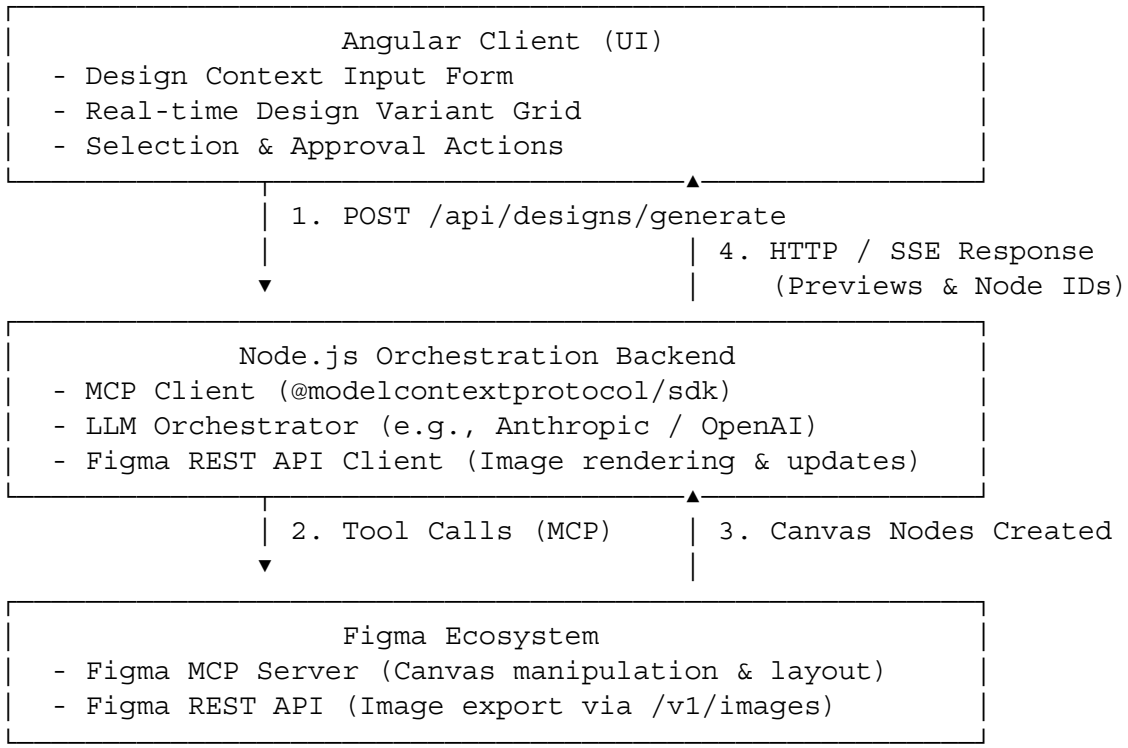
Figma MCP Integration with Angular: Architecture & Boilerplate

This document outlines the end-to-end architecture, implementation plan, and boilerplate code to generate UI design variants using the Figma Model Context Protocol (MCP) server, preview them inside an Angular application, and facilitate user review and approval.

1. System Architecture

Browsers cannot connect directly to MCP servers due to protocol constraints (SSE/stdio transport) and security considerations (Figma Personal Access Tokens and LLM API keys must remain secure). A dedicated backend orchestration service acts as the intermediary.



2. End-to-End Workflow

1. **Context Submission:** The Angular user specifies layout rules, screen requirements, theme variables, or target audience details.
2. **Variant Generation:** The Node.js backend prompts an LLM configured with Figma MCP tools. For each variant:
    - The agent calls the Figma MCP server to construct nodes/frames directly on the designated Figma canvas.

    - The MCP returns the created node_ids.
3. **Snapshot Extraction:** The backend queries Figma's REST API (GET /v1/images/:file_key?ids=...) to convert the newly created node_ids into high-resolution PNG/SVG preview URLs.
4. **Interactive Review:** Angular receives the list of variants, rendering responsive cards with preview images or embedded Figma frames.
5. **Approval Action:** The user picks a design. Angular sends the selected node_id to the backend, which moves or tags the node in Figma (e.g., moving to an "Approved" page or assigning status metadata).

3. Node.js Backend Boilerplate

Prerequisites & Dependencies

```
mkdir figma-mcp-backend && cd figma-mcp-backend
npm init -y
npm install express dotenv cors @modelcontextprotocol/sdk
@anthropic-ai/sdk axios
npm install --save-dev typescript @types/node @types/express
@types/cors ts-node
npx tsc --init
```

Environment Configuration (.env)

```
PORT=3000
FIGMA_ACCESS_TOKEN=figd_your_personal_access_token_here
FIGMA_FILE_KEY=your_figma_file_key_here
ANTHROPIC_API_KEY=your_anthropic_key_here
FIGMA_MCP_SERVER_URL=https://mcp.figma.com/mcp
```

Server Implementation (server.ts)

```
import express, { Request, Response } from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import axios from 'axios';
import Anthropic from '@anthropic-ai/sdk';
import { Client } from '@modelcontextprotocol/sdk/client/index.js';
import { SSEClientTransport } from
'@modelcontextprotocol/sdk/client/sse.js';

dotenv.config();

const app = express();
app.use(cors());
```

```
app.use(express.json());

const FIGMA_TOKEN = process.env.FIGMA_ACCESS_TOKEN!;
const FIGMA_FILE_KEY = process.env.FIGMA_FILE_KEY!;
const anthropic = new Anthropic({ apiKey:
process.env.ANTHROPIC_API_KEY });

// 1. Initialize MCP Client
async function getFigmaMcpClient(): Promise<Client> {
  const transport = new SSEClientTransport(
    new URL(process.env.FIGMA_MCP_SERVER_URL ||
'https://mcp.figma.com/mcp'),
    {
      eventSourceInit: {
        headers: {
          Authorization: `Bearer ${FIGMA_TOKEN}`
        }
      }
    }
  );

  const client = new Client(
    { name: 'angular-figma-orchestrator', version: '1.0.0' },
    { capabilities: {} }
  );

  await client.connect(transport);
  return client;
}

// 2. Fetch rendered PNG snapshots from Figma REST API
async function fetchNodeSnapshots(fileKey: string, nodeIds: string[]):
Promise<Record<string, string>> {
  const idsParam = nodeIds.join(',');
  const response = await axios.get(

`https://api.figma.com/v1/images/${fileKey}?ids=${idsParam}&format=png
&scale=2`,
    {
      headers: {
        'X-Figma-Token': FIGMA_TOKEN
      }
    }
  );
  return response.data.images; // Map of { [nodeId]: "https://..." }
}

// 3. Endpoint: Generate Design Variants
```

```
app.post('/api/designs/generate', async (req: Request, res: Response)
=> {
  try {
    const { context, variantCount = 2 } = req.body;
    const mcpClient = await getFigmaMcpClient();

    // Query tools available on the Figma MCP server
    const { tools } = await mcpClient.listTools();

    const prompt = `
      Context: ${context}
      File Key: ${FIGMA_FILE_KEY}
      Task: Generate ${variantCount} distinct UI layout options on the
Figma canvas.
      Return the node IDs of the generated top-level frames.
    `;

    // Agent handles tool selection and canvas generation
    const response = await anthropic.messages.create({
      model: 'claude-3-5-sonnet-20241022',
      max_tokens: 4096,
      tools: tools.map((t) => ({
        name: t.name,
        description: t.description || '',
        input_schema: t.inputSchema as Anthropic.Tool.InputSchema
      })),
      messages: [{ role: 'user', content: prompt }]
    });

    // Execute tool use calls against the MCP client
    const generatedNodeIds: { id: string; name: string }[] = [];
    for (const content of response.content) {
      if (content.type === 'tool_use') {
        const toolResult = await mcpClient.callTool({
          name: content.name,
          arguments: content.input as Record<string, unknown>
        });

        // Parse result payload from MCP response (adjust based on
tool return schema)
        if (toolResult && toolResult.content) {
          // Extract created frame IDs
        }
      }
    }

    // Fallback/Mock IDs for preview if testing locally
    const mockIds = ['1:100', '1:200'];
```

```
    const activeIds = generatedNodeIds.length ? generatedNodeIds.map(n
=> n.id) : mockIds;

    // Retrieve live preview images for the Angular UI
    const imagesMap = await fetchNodeSnapshots(FIGMA_FILE_KEY,
activeIds);

    const variants = activeIds.map((id, index) => ({
      id: `var-${index + 1}`,
      nodeId: id,
      name: `Option ${index + 1}`,
      previewUrl: imagesMap[id] || '',
      embedUrl:
`https://embed.figma.com/design/${FIGMA_FILE_KEY}?node-id=${id}&embed-
host=my-angular-app`
    }));

    res.json({ success: true, variants });
  } catch (error: any) {
    console.error('Error generating designs:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// 4. Endpoint: Approve Selected Variant
app.post('/api/designs/approve', async (req: Request, res: Response)
=> {
  try {
    const { nodeId, comments } = req.body;

    // Example post-approval action: Post a comment or update file
metadata
    await axios.post(
      `https://api.figma.com/v1/files/${FIGMA_FILE_KEY}/comments`,
      {
        message: `Design variant approved: ${comments || 'No
comments'}`,
        client_meta: { node_id: nodeId }
      },
      {
        headers: { 'X-Figma-Token': FIGMA_TOKEN }
      }
    );

    res.json({ success: true, message: `Node ${nodeId} successfully
approved.` });
  } catch (error: any) {
    console.error('Error approving design:', error);
```

```
    res.status(500).json({ success: false, error: error.message });
  }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Figma MCP Orchestration Server listening on port
${PORT}`);
});
```

4. Angular Client Implementation

Design Service (src/app/services/figma-design.service.ts)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface DesignVariant {
  id: string;
  nodeId: string;
  name: string;
  previewUrl: string;
  embedUrl: string;
}

export interface GenerateResponse {
  success: boolean;
  variants: DesignVariant[];
}

@Injectable({
  providedIn: 'root'
})
export class FigmaDesignService {
  private apiUrl = 'http://localhost:3000/api/designs';

  constructor(private http: HttpClient) {}

  generateVariants(context: string, variantCount: number = 2):
Observable<GenerateResponse> {
    return this.http.post<GenerateResponse>(`${this.apiUrl}/generate`,
{
      context,
      variantCount
    });
```

```
  }

  approveDesign(nodeId: string, comments?: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/approve`, { nodeId, comments
});
  }
}
```

Reviewer Component (src/app/components/design-reviewer.component.ts)

```
import { Component, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { DomSanitizer, SafeResourceUrl } from
'@angular/platform-browser';
import { FigmaDesignService, DesignVariant } from
'../services/figma-design.service';

@Component({
  selector: 'app-design-reviewer',
  standalone: true,
  imports: [CommonModule, FormsModule],
  template: `
    <div class="container">
      <header class="header">
        <h1>AI Design Review Studio</h1>
        <p>Define requirements, generate Figma variants, and approve
the best layout.</p>
      </header>

      <!-- Context Input Section -->
      <section class="prompt-box">
        <label for="contextInput">Design Context &
Requirements</label>
        <textarea
          id="contextInput"
          [(ngModel)]="contextPrompt"
          rows="3"
          placeholder="e.g., E-commerce product details screen, clean
minimalist layout, high emphasis on call-to-action button and user
ratings."
        ></textarea>

        <div class="actions-bar">
          <button (click)="generate()" [disabled]="isLoading() ||
```

```
!contextPrompt.trim()">
            {{ isLoading() ? 'Generating Variants...' : 'Generate
Figma Designs' }}
          </button>
        </div>
      </section>

      <!-- Variant Selection Gallery -->
      <section *ngIf="variants().length > 0" class="variants-gallery">
        <h2>Select a Design Variant</h2>
        <div class="grid">
          <div
            *ngFor="let variant of variants()"
            class="variant-card"
            [class.selected]="selectedVariant()?.id === variant.id"
            (click)="selectVariant(variant)"
          >
            <div class="card-media">
              <img
                *ngIf="variant.previewUrl"
                [src]="variant.previewUrl"
                [alt]="variant.name"
                class="preview-img"
              />
              <div *ngIf="!variant.previewUrl" class="no-preview">
                Preview Unavailable
              </div>
            </div>

            <div class="card-info">
              <h3>{{ variant.name }}</h3>
              <p>Node ID: <code>{{ variant.nodeId }}</code></p>
            </div>
          </div>
        </div>

        <!-- Approval Box -->
        <div *ngIf="selectedVariant()" class="approval-panel">
          <h3>Confirm Selection</h3>
          <p>You have selected: <strong>{{ selectedVariant()?.name
}}</strong></p>
          <button class="approve-btn" (click)="confirmApproval()"
[disabled]="isSubmitting()">
            {{ isSubmitting() ? 'Approving...' : 'Approve This Design'
}}
          </button>
        </div>
      </section>
```

```
    </div>
  `,
  styles: [`
    .container { max-width: 1100px; margin: 0 auto; padding: 2rem;
font-family: sans-serif; }
    .header { margin-bottom: 2rem; }
    .prompt-box { background: #f8fafc; border: 1px solid #e2e8f0;
border-radius: 8px; padding: 1.5rem; }
    .prompt-box textarea { width: 100%; border: 1px solid #cbd5e1;
border-radius: 6px; padding: 0.75rem; font-size: 1rem; }
    .actions-bar { margin-top: 1rem; display: flex; justify-content:
flex-end; }

    button { background: #0284c7; color: white; border: none; padding:
0.65rem 1.25rem; border-radius: 6px; cursor: pointer; font-weight:
600; }
    button:disabled { background: #94a3b8; cursor: not-allowed; }

    .variants-gallery { margin-top: 2.5rem; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit,
minmax(300px, 1fr)); gap: 1.5rem; margin-top: 1rem; }

    .variant-card { border: 2px solid #e2e8f0; border-radius: 8px;
overflow: hidden; cursor: pointer; transition: all 0.2s ease;
background: #fff; }
    .variant-card:hover { transform: translateY(-2px); box-shadow: 0
4px 6px -1px rgba(0, 0, 0, 0.1); }
    .variant-card.selected { border-color: #0284c7; box-shadow: 0 0 0
3px rgba(2, 132, 199, 0.2); }

    .card-media { width: 100%; height: 220px; background: #f1f5f9;
display: flex; align-items: center; justify-content: center; overflow:
hidden; }
    .preview-img { width: 100%; height: 100%; object-fit: cover; }
    .card-info { padding: 1rem; }
    .card-info h3 { margin: 0 0 0.25rem 0; font-size: 1.1rem; }
    .card-info code { background: #f1f5f9; padding: 0.2rem 0.4rem;
border-radius: 4px; font-size: 0.85rem; }

    .approval-panel { margin-top: 2rem; padding: 1.5rem; background:
#f0fdf4; border: 1px solid #bbf7d0; border-radius: 8px; display: flex;
align-items: center; justify-content: space-between; }
    .approve-btn { background: #16a34a; }
  `]
})
export class DesignReviewerComponent {
  contextPrompt = '';
  variants = signal<DesignVariant[]>([]);
```

```
  selectedVariant = signal<DesignVariant | null>(null);
  isLoading = signal(false);
  isSubmitting = signal(false);

  constructor(
    private designService: FigmaDesignService,
    private sanitizer: DomSanitizer
  ) {}

  generate() {
    this.isLoading.set(true);
    this.selectedVariant.set(null);


this.designService.generateVariants(this.contextPrompt).subscribe({
      next: (res) => {
        this.variants.set(res.variants);
        this.isLoading.set(false);
      },
      error: (err) => {
        console.error('Generation failed:', err);
        this.isLoading.set(false);
      }
    });
  }

  selectVariant(variant: DesignVariant) {
    this.selectedVariant.set(variant);
  }

  confirmApproval() {
    const selected = this.selectedVariant();
    if (!selected) return;

    this.isSubmitting.set(true);
    this.designService.approveDesign(selected.nodeId).subscribe({
      next: () => {
        alert(`Design "${selected.name}" has been approved!`);
        this.isSubmitting.set(false);
      },
      error: (err) => {
        console.error('Approval failed:', err);
        this.isSubmitting.set(false);
      }
    });
  }

  getSafeEmbedUrl(url: string): SafeResourceUrl {
```

```
    return this.sanitizer.bypassSecurityTrustResourceUrl(url);
  }
}
```

5. Summary & Best Practices

1. **Snapshots vs. Iframes:** Prefer rendering static PNG snapshots via the /v1/images REST endpoint rather than multiple concurrent iframes. Iframes incur performance overhead and require the user to have active session credentials for Figma in their browser.
2. **Asynchronous Processing:** If LLM layout generation takes over 15 seconds, convert the POST /api/designs/generate endpoint to return a job ID and stream events via Server-Sent Events (SSE) or WebSockets to keep the Angular UI responsive.
3. **Canvas Segregation:** Instruct the Figma MCP client to generate variants on a designated "Staging / AI Drafts" page inside the Figma file, preventing clutter in core design production files until a user officially approves a candidate frame.